
EngTeam Migration — Macro Archive

One-shot build macros saved here for reference. None of these need to live in the workbook day-to-day. Copy back into a VBA module if you ever need to re-run.

Why this exists: these macros either (a) were one-time build operations during the Option 3 migration, (b) reference the old structure (AR sheet, checkboxes, fragmented CF) and don't apply to the current file, or (c) are utilities that might be useful later but aren't worth carrying as live code. Module 1 in the workbook holds only what's used regularly. Module 2 holds rare-but-still-relevant macros (BuildChangelogTab, BuildHowToUseTab, NewWeek, BuildRiskRegisterTab, PopulateRiskRegister).

SECTIONS

- Build macros — used during Option 3 migration (already run, archived)
- Old structure macros — for reference only (do NOT run on current file)
- Utilities — occasional one-off helpers

Build macros — used during Option 3 migration

All of these have already done their job during the migration phases. Kept here in case the build ever needs to be redone from scratch.

DeleteAllCheckboxes

Phase 2 — strips all 257 form-control checkboxes from MM. Used during initial cleanup.

PHASE 2 — DONE

```
Sub DeleteAllCheckboxes()  
    Dim ws As Worksheet  
    Set ws = ThisWorkbook.Sheets("Meeting Minutes")  
    Dim shp As Shape  
    Dim count As Long  
    count = 0  
    Application.ScreenUpdating = False  
    For Each shp In ws.Shapes  
        If shp.Type = msoFormControl Then  
            If shp.FormControlType = xlCheckBox Then  
                shp.Delete  
                count = count + 1  
            End If  
        End If  
    Next shp  
    Application.ScreenUpdating = True  
    MsgBox count & " checkboxes deleted from Meeting Minutes."  
End Sub
```

DeleteBrokenCF

Phase 2 — deletes any CF rule whose formula contains #REF!.

PHASE 2 — DONE

```
Sub DeleteBrokenCF()  
    Dim ws As Worksheet  
    Set ws = ThisWorkbook.Sheets("Meeting Minutes")  
    Dim fc As FormatCondition  
    Dim deleted As Long  
    Dim i As Long  
    Application.ScreenUpdating = False  
    For i = ws.Cells.FormatConditions.Count To 1 Step -1  
        Set fc = ws.Cells.FormatConditions(i)  
        If InStr(fc.Formula1, "#REF!") > 0 Then  
            fc.Delete  
            deleted = deleted + 1  
        End If  
    Next i  
    Application.ScreenUpdating = True  
    MsgBox deleted & " broken CF rules deleted."  
End Sub
```

DefragmentCF + AddCellIs + CheckCFAfterDefrag

Phase 2 — rebuilt fragmented Status/Priority CF rules onto contiguous ranges.

PHASE 2 – DONE

```
Sub DefragmentCF()
    Dim ws As Worksheet
    Set ws = ThisWorkbook.Sheets("Meeting Minutes")
    Application.ScreenUpdating = False
    Dim fc As FormatCondition
    Dim i As Long
    Dim deleted As Long
    For i = ws.Cells.FormatConditions.Count To 1 Step -1
        Set fc = ws.Cells.FormatConditions(i)
        Dim sq As String
        sq = fc.AppliesTo.Address
        If InStr(sq, ",") > 0 Then
            If InStr(sq, "$H$") > 0 Or InStr(sq, "$I$") > 0 Then
                fc.Delete
                deleted = deleted + 1
            End If
        End If
    Next i
    Dim rngP As Range
    Set rngP = ws.Range("I16:I1000")
    AddCellIs rngP, "Critical", RGB(192, 0, 0), RGB(255, 255, 255)
    AddCellIs rngP, "High", RGB(237, 125, 49), RGB(255, 255, 255)
    AddCellIs rngP, "Medium", RGB(255, 192, 0), RGB(26, 26, 46)
    AddCellIs rngP, "Low", RGB(166, 166, 166), RGB(255, 255, 255)
    Dim rngS As Range
    Set rngS = ws.Range("H16:H1000")
    AddCellIs rngS, "Open", RGB(221, 235, 247), RGB(31, 78, 121)
    AddCellIs rngS, "In Progress", RGB(255, 242, 204), RGB(128, 96, 0)
    AddCellIs rngS, "Done", RGB(226, 239, 218), RGB(55, 86, 35)
    AddCellIs rngS, "On Hold", RGB(251, 229, 214), RGB(132, 60, 12)
    AddCellIs rngS, "Waiting", RGB(242, 232, 251), RGB(91, 42, 135)
    Application.ScreenUpdating = True
    MsgBox "Defrag complete. Deleted " & deleted & " fragmented CF rules."
End Sub

Sub AddCellIs(rng As Range, value As String, fillColor As Long, fontColor As Long)
    With rng.FormatConditions.Add(Type:=xlExpression, _
        Formula1:="=" & rng.Cells(1, 1).Address(False, True) & "=" & value & "")
        .Interior.Pattern = xlSolid
        .Interior.Color = fillColor
        .Font.Color = fontColor
        .Font.Bold = True
        .StopIfTrue = False
    End With
End Sub
```

End Sub

```
Sub CheckCFAfterDefrag()  
    Dim ws As Worksheet  
    Set ws = ThisWorkbook.Sheets("Meeting Minutes")  
    Dim msg As String, i As Long, fc As FormatCondition  
    msg = "CF rules on Meeting Minutes:" & vbCrLf & vbCrLf  
    For i = 1 To ws.Cells.FormatConditions.Count  
        Set fc = ws.Cells.FormatConditions(i)  
        Dim sq As String: sq = fc.AppliesTo.Address  
        Dim frags As Long: frags = UBound(Split(sq, ",")) + 1  
        Dim flag As String: flag = IIf(frags = 1, "OK", "FRAG")  
        msg = msg & i & ". [" & flag & "]" & Left(sq, 40) & vbCrLf  
        msg = msg & "    " & Left(fc.Formula1, 60) & vbCrLf  
    Next i  
    MsgBox msg  
End Sub
```

BuildProjectRegister + AddLifecycleCF

Phase 3 — created the Project Register sheet and harvested project names from MM col M.

PHASE 3 — DONE

```
Sub BuildProjectRegister()
    Dim ws As Worksheet
    Application.DisplayAlerts = False
    On Error Resume Next
    ThisWorkbook.Sheets("Project Register").Delete
    On Error GoTo 0
    Application.DisplayAlerts = True
    Set ws = ThisWorkbook.Sheets.Add(After:=ThisWorkbook.Sheets("Summary"))
    ws.Name = "Project Register"
    ws.Range("A1:G1").Value = Array("Project", "Lifecycle", "PM", "Budget", "PC date", "D
    With ws.Range("A1:G1")
        .Font.Name = "Arial"
        .Font.Size = 10
        .Font.Bold = True
        .Font.Color = RGB(255, 255, 255)
        .Interior.Pattern = xlSolid
        .Interior.Color = RGB(31, 56, 100)
        .HorizontalAlignment = xlLeft
    End With
    ws.Rows(1).RowHeight = 22
    ws.Columns("A").ColumnWidth = 28
    ws.Columns("B").ColumnWidth = 14
    ws.Columns("C").ColumnWidth = 9
    ws.Columns("D").ColumnWidth = 12
    ws.Columns("E").ColumnWidth = 12
    ws.Columns("F").ColumnWidth = 12
    ws.Columns("G").ColumnWidth = 50
    With ws.Range("B2:B100").Validation
        .Delete
        .Add Type:=xlValidateList, Formula1:="Active,DLP,Future,Closed,Strategic,General"
        .IgnoreBlank = True
        .InCellDropdown = True
    End With
    ws.Range("E2:F100").NumberFormat = "dd/mm/yyyy"
    AddLifecycleCF ws.Range("B2:B100"), "Active", RGB(221, 235, 247), RGB(31, 78, 121)
    AddLifecycleCF ws.Range("B2:B100"), "DLP", RGB(225, 245, 238), RGB(15, 110, 86)
    AddLifecycleCF ws.Range("B2:B100"), "Future", RGB(221, 235, 247), RGB(24, 95, 165)
    AddLifecycleCF ws.Range("B2:B100"), "Closed", RGB(241, 239, 232), RGB(95, 94, 90)
    AddLifecycleCF ws.Range("B2:B100"), "Strategic", RGB(238, 237, 254), RGB(83, 74, 183)
    AddLifecycleCF ws.Range("B2:B100"), "General", RGB(241, 239, 232), RGB(68, 68, 65)
    Dim mm As Worksheet
    Set mm = ThisWorkbook.Sheets("Meeting Minutes")
    Dim projects As Object
    Set projects = CreateObject("Scripting.Dictionary")
```

```

Dim r As Long
For r = 16 To mm.Cells(mm.Rows.Count, 13).End(xlUp).Row
    Dim p As Variant
    p = mm.Cells(r, 13).Value
    If Not IsEmpty(p) And Not IsError(p) Then
        Dim pStr As String
        pStr = Trim(CStr(p))
        If Len(pStr) > 0 And InStr(pStr, "#REF!") = 0 And InStr(pStr, "=IF") = 0 Then
            If Not projects.exists(pStr) Then
                projects.Add pStr, True
            End If
        End If
    End If
Next r
Dim outRow As Long: outRow = 2
Dim key As Variant
For Each key In projects.keys
    ws.Cells(outRow, 1).Value = key
    outRow = outRow + 1
Next key
ws.Activate
ws.Range("A2").Select
ActiveWindow.FreezePanels = True
MsgBox projects.Count & " projects harvested."
End Sub

Sub AddLifecycleCF(rng As Range, value As String, fillColor As Long, fontColor As Long)
    With rng.FormatConditions.Add(Type:=xlExpression, _
        Formula1:="=" & rng.Cells(1, 1).Address(False, True) & "="" & value & """)
        .Interior.Pattern = xlSolid
        .Interior.Color = fillColor
        .Font.Color = fontColor
        .Font.Bold = True
        .StopIfTrue = False
    End With
End Sub

```

AppendNonProjects

Phase 3 — appended General/Team/Strategic items to Project Register and updated Lifecycle dropdown to 7 values.

PHASE 3 — DONE

```
Sub AppendNonProjects()  
    Dim ws As Worksheet  
    Set ws = ThisWorkbook.Sheets("Project Register")  
    With ws.Range("B2:B100").Validation  
        .Delete  
        .Add Type:=xlValidateList, Formula1:="Active,DLP,Future,Closed,Team,Strategic,Gen  
        .IgnoreBlank = True  
        .InCellDropdown = True  
    End With  
    AddLifecycleCF ws.Range("B2:B100"), "Team", RGB(241, 239, 232), RGB(68, 68, 65)  
    Dim nextRow As Long  
    nextRow = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row + 1  
    Dim items As Variant  
    items = Array( _  
        Array("Safety Moment", "Team"), _  
        Array("Team Commitments", "Team"), _  
        Array("Team Improvements", "Team"), _  
        Array("Resourcing", "Team"), _  
        Array("Project Reporting", "Team"), _  
        Array("CRMs", "General"), _  
        Array("PM Framework", "Strategic"), _  
        Array("Training & Development", "Strategic"), _  
        Array("Events", "General"), _  
        Array("Leave", "General"))  
    Dim i As Long  
    For i = 0 To UBound(items)  
        ws.Cells(nextRow + i, 1).Value = items(i)(0)  
        ws.Cells(nextRow + i, 2).Value = items(i)(1)  
    Next i  
    MsgBox UBound(items) + 1 & " rows appended."  
End Sub
```


ClearColAData

Phase 4 — unmerged all banner rows in MM col A and cleared col A data, leaving rows 1-14 untouched.

PHASE 4 — DONE

```
Sub ClearColAData()  
    Dim ws As Worksheet  
    Set ws = ThisWorkbook.Sheets("Meeting Minutes")  
    Dim lastRow As Long  
    lastRow = ws.UsedRange.Rows.Count  
    Application.ScreenUpdating = False  
    Dim cell As Range  
    Dim unmerged As Long  
    For Each cell In ws.Range("A16:A" & lastRow)  
        If cell.MergeCells Then  
            cell.MergeArea.UnMerge  
            unmerged = unmerged + 1  
        End If  
    Next cell  
    ws.Range("A16:A" & lastRow).ClearContents  
    Application.ScreenUpdating = True  
    MsgBox "Unmerged " & unmerged & " banners. Cleared col A 16-" & lastRow & "."  
End Sub
```

FixODDaysFormulas

Phase 5 — cleared all #REF! formulas from MM col G (OD Days). Cleared 292 cells.

PHASE 5 — DONE

```
Sub FixODDaysFormulas()  
    Dim ws As Worksheet  
    Set ws = ThisWorkbook.Sheets("Meeting Minutes")  
    Dim lastRow As Long  
    lastRow = ws.UsedRange.Rows.Count  
    Application.ScreenUpdating = False  
    Dim cleared As Long  
    Dim cell As Range  
    For Each cell In ws.Range("G16:G" & lastRow)  
        If Not cell.MergeCells Then  
            Dim hasRef As Boolean  
            hasRef = False  
            On Error Resume Next  
            If InStr(CStr(cell.Formula), "#REF!") > 0 Then hasRef = True  
            On Error GoTo 0  
            If Not hasRef Then  
                If IsError(cell.Value) Then hasRef = True  
            End If  
            If hasRef Then  
                cell.ClearContents  
                cleared = cleared + 1  
            End If  
        End If  
    Next cell  
    Application.ScreenUpdating = True  
    MsgBox "Cleared " & cleared & " #REF! cells in col G."  
End Sub
```

InsertProjectBanners

Phase 6 — walked col I and inserted a Project banner row each time the project changed.

PHASE 6 – DONE

```
Sub InsertProjectBanners()  
    Dim mm As Worksheet  
    Set mm = ThisWorkbook.Sheets("Meeting Minutes")  
    Application.ScreenUpdating = False  
    Dim lastRow As Long  
    lastRow = mm.Cells(mm.Rows.Count, 9).End(xlUp).Row  
    Dim r As Long  
    Dim prevProj As String  
    prevProj = ""  
    Dim inserted As Long  
    r = 16  
    Do While r <= lastRow  
        Dim curProj As String  
        curProj = Trim(CStr(mm.Cells(r, 9).Value))  
        If curProj <> "" And curProj <> prevProj Then  
            mm.Rows(r).Insert Shift:=xlDown  
            mm.Range("A" & r & ":J" & r).Merge  
            With mm.Cells(r, 1)  
                .Value = curProj  
                .Font.Bold = True  
                .Font.Color = RGB(4, 44, 83)  
                .Interior.Color = RGB(181, 212, 244)  
                .HorizontalAlignment = xlLeft  
                .IndentLevel = 2  
            End With  
            inserted = inserted + 1  
            prevProj = curProj  
            r = r + 2  
            lastRow = lastRow + 1  
        Else  
            r = r + 1  
        End If  
    Loop  
    Application.ScreenUpdating = True  
    MsgBox inserted & " project banners inserted."  
End Sub
```

InsertLifecycleBanners + LCColor

Phase 6 — read Project Register and inserted Lifecycle banners above Project banners whenever lifecycle changed.

PHASE 6 – DONE

```

Sub InsertLifecycleBanners()
    Dim mm As Worksheet, pr As Worksheet
    Set mm = ThisWorkbook.Sheets("Meeting Minutes")
    Set pr = ThisWorkbook.Sheets("Project Register")
    Application.ScreenUpdating = False
    Dim lcMap As Object
    Set lcMap = CreateObject("Scripting.Dictionary")
    Dim lastPR As Long
    lastPR = pr.Cells(pr.Rows.Count, 1).End(xlUp).Row
    Dim pr_r As Long
    For pr_r = 2 To lastPR
        Dim pName As String, pLC As String
        pName = Trim(CStr(pr.Cells(pr_r, 1).Value))
        pLC = Trim(CStr(pr.Cells(pr_r, 2).Value))
        If pName <> "" Then lcMap(pName) = pLC
    Next pr_r
    Dim lastRow As Long
    lastRow = mm.Cells(mm.Rows.Count, 9).End(xlUp).Row
    Dim r As Long
    Dim prevLC As String
    prevLC = ""
    Dim inserted As Long
    r = 16
    Do While r <= lastRow
        If mm.Cells(r, 1).MergeCells Then
            r = r + 1
        Else
            Dim curProj As String
            curProj = Trim(CStr(mm.Cells(r, 9).Value))
            If curProj = "" Then
                r = r + 1
            Else
                Dim curLC As String
                If lcMap.exists(curProj) Then
                    curLC = CStr(lcMap(curProj))
                Else
                    curLC = "MISC"
                End If
                If curLC = "" Then curLC = "MISC"
                If curLC <> prevLC Then
                    Dim insertAt As Long
                    insertAt = r - 1
                    If mm.Cells(insertAt, 1).MergeCells Then
                        mm.Rows(insertAt).Insert Shift:=xlDown
                    End If
                End If
            End If
        End If
    Loop
End Sub

```

```

        mm.Range("A" & insertAt & ":J" & insertAt).Merge
        With mm.Cells(insertAt, 1)
            .Value = UCase(curlC)
            .Font.Bold = True
            .Font.Color = RGB(255, 255, 255)
            .Interior.Color = LCColor(curlC)
            .HorizontalAlignment = xlLeft
            .IndentLevel = 1
        End With
        inserted = inserted + 1
        r = r + 1
        lastRow = lastRow + 1
    End If
    prevLC = curlC
End If
r = r + 1
End If
End If
Loop
Application.ScreenUpdating = True
MsgBox inserted & " lifecycle banners inserted."
End Sub

Function LCColor(lc As String) As Long
    Select Case lc
        Case "Team":      LCColor = RGB(68, 68, 65)
        Case "Active":    LCColor = RGB(31, 56, 100)
        Case "DLP":       LCColor = RGB(15, 110, 86)
        Case "Future":    LCColor = RGB(24, 95, 165)
        Case "Strategic":  LCColor = RGB(83, 74, 183)
        Case "General":    LCColor = RGB(95, 94, 90)
        Case "Closed":     LCColor = RGB(95, 94, 90)
        Case Else:        LCColor = RGB(180, 178, 169)
    End Select
End Function

```

RebuildDropdowns + AddDV

Phase 7 — wiped fragmented dropdowns and rebuilt fresh on contiguous ranges. Type = Action,Risk only.

PHASE 7 — DONE

```
Sub RebuildDropdowns()  
    Dim mm As Worksheet  
    Set mm = ThisWorkbook.Sheets("Meeting Minutes")  
    Application.ScreenUpdating = False  
    mm.Range("A16:J1000").Validation.Delete  
    AddDV mm, "A16:A1000", "Action,Risk"  
    AddDV mm, "C16:C1000", "TM1,TM2,TM3,TM4,TM5,TM6,ALL,AL"  
    AddDV mm, "E16:E1000", "Open,In Progress,Done,On Hold,Waiting"  
    AddDV mm, "F16:F1000", "Critical,High,Medium,Low"  
    Application.ScreenUpdating = True  
    MsgBox "Dropdowns rebuilt."  
End Sub  
  
Sub AddDV(ws As Worksheet, rangeStr As String, listValues As String)  
    With ws.Range(rangeStr).Validation  
        .Delete  
        .Add Type:=xlValidateList, Formula1:=listValues  
        .IgnoreBlank = True  
        .InCellDropdown = True  
    End With  
End Sub
```

WipeAllCFOnMM + AddCleanCF

Phase 7 — wiped all CF on MM and rebuilt 12 clean rules on contiguous ranges. Fills set manually after.

PHASE 7 — DONE

```
Sub WipeAllCFOnMM()  
    Dim ws As Worksheet  
    Set ws = ThisWorkbook.Sheets("Meeting Minutes")  
    ws.Cells.FormatConditions.Delete  
    MsgBox "All CF rules wiped from MM."  
End Sub  
  
Sub AddCleanCF()  
    Dim ws As Worksheet  
    Set ws = ThisWorkbook.Sheets("Meeting Minutes")  
    ws.Range("A16:J1000").FormatConditions.Add Type:=xlExpression, _  
        Formula1:="=AND($A16=""Action"", $D16<>"", $D16
```

CleanDataAreaFormatting

Phase 7 — cleared fills from non-banner data rows and applied soft borders. Used to tidy up before final colours.

PHASE 7 — DONE

```
Sub CleanDataAreaFormatting()  
    Dim ws As Worksheet  
    Set ws = ThisWorkbook.Sheets("Meeting Minutes")  
    Application.ScreenUpdating = False  
    Dim lastRow As Long  
    lastRow = ws.UsedRange.Rows.Count  
    If lastRow < 16 Then lastRow = 16  
    Dim cleared As Long  
    Dim r As Long  
    For r = 16 To lastRow  
        If Not ws.Cells(r, 1).MergeCells Then  
            ws.Range("A" & r & ":J" & r).Interior.Pattern = xlNone  
            cleared = cleared + 1  
        End If  
    Next r  
    Dim dataArea As Range  
    Set dataArea = ws.Range("A15:J" & lastRow)  
    With dataArea.Borders  
        .LineStyle = xlContinuous  
        .Weight = xlThin  
        .Color = RGB(212, 212, 212)  
    End With  
    Application.ScreenUpdating = True  
    MsgBox "Cleared fills on " & cleared & " data rows."  
End Sub
```

Old structure macros — for reference only

These were built for the OLD template (separate Action Register sheet, checkboxes, two-way sync). They will NOT work on the current Option 3 file. Kept for archival reference only.

PopulateActionRegister

Old AR sync. Read MM checkboxes, wrote actions to AR. Replaced by AR view filter on MM.

OLD STRUCTURE – DO NOT RUN

```
Sub PopulateActionRegister()
    Dim wsMin As Worksheet
    Dim wsAR As Worksheet
    Dim r As Long
    Dim arRow As Long
    Dim arKeys As Object
    Dim mmKey As String
    Dim added As Long
    Set wsMin = ThisWorkbook.Sheets("Meeting Minutes")
    Set wsAR = ThisWorkbook.Sheets("Action Register")
    Set arKeys = CreateObject("Scripting.Dictionary")
    added = 0
    For arRow = 4 To 103
        If Trim(CStr(wsAR.Cells(arRow, 2).Value)) <> "" Then
            arKeys(Trim(CStr(wsAR.Cells(arRow, 2).Value)) & "|" & _
                Trim(CStr(wsAR.Cells(arRow, 3).Value))) = True
        End If
    Next arRow
    arRow = 4
    Do While Trim(CStr(wsAR.Cells(arRow, 2).Value)) <> "" And arRow <= 103
        arRow = arRow + 1
    Loop
    For r = 15 To 300
        If wsMin.Cells(r, 1).MergeCells Then GoTo nextRow
        If wsMin.Cells(r, 3).Value <> True Then GoTo nextRow
        If Trim(CStr(wsMin.Cells(r, 2).Value)) = "" Then GoTo nextRow
        mmKey = Trim(CStr(wsMin.Cells(r, 2).Value)) & "|" & _
            Trim(CStr(wsMin.Cells(r, 6).Value))
        If arKeys.exists(mmKey) Then GoTo nextRow
        If arRow > 103 Then Exit For
        wsAR.Cells(arRow, 1).Value = wsMin.Cells(r, 12).Value
        wsAR.Cells(arRow, 2).Value = wsMin.Cells(r, 2).Value
        wsAR.Cells(arRow, 3).Value = wsMin.Cells(r, 6).Value
        wsAR.Cells(arRow, 4).Value = wsMin.Cells(r, 7).Value
        wsAR.Cells(arRow, 5).Value = wsMin.Cells(r, 9).Value
        wsAR.Cells(arRow, 6).Value = wsMin.Cells(r, 8).Value
        wsAR.Cells(arRow, 7).Value = Date
        wsAR.Cells(arRow, 8).Value = wsMin.Cells(r, 11).Value
        arKeys(mmKey) = True
        arRow = arRow + 1
        added = added + 1
    Next r
End Sub
```

```
nextRow:
  Next r
End Sub
```

SyncStatusBack, PushNewActionsToMinutes, EndMeeting, SyncAll

Old AR↔MM sync logic — caused the data-overwrite bug that triggered this whole migration. Replaced entirely by AR-as-filter-view.

OLD STRUCTURE – DESTRUCTIVE ON CURRENT FILE

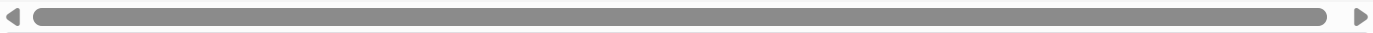
```
' Replaced by Option 3 architecture.
' Old code preserved in earlier file Migration_v1_P1_complete.xlsm if ever needed.
' Do NOT paste back into current file.
```

AddSection, AddSubsection

Old banner-creation macros. Banners now driven by Project Register / RebuildBanners.

OLD STRUCTURE

```
' Replaced by InsertProjectBanners + InsertLifecycleBanners (run via macros above).
' For ongoing use, RebuildBanners macro (when written) handles all banner regeneration.
```



AddCheckboxes, HideTrueFalseText, AddAttendeeCheckboxes, ReplaceAttendeeCheckboxesWithDropdown

Old checkbox-management macros. All checkboxes deleted in Phase 2 and not used in current template.

OLD STRUCTURE

```
' Checkboxes removed in Phase 2. Type = dropdown column now (Action / Risk / blank).  
' Attendees use Yes/No dropdown instead of checkboxes.
```

FormatMM, AddCF, FixOverdueCF, AlternateRowShading, FixPriorityStatusCF, RebuildMMHeader

Old format-rebuild macros. Reference old column positions (Action col C, Owner col F etc). Replaced by new structure CF/dropdown macros.

OLD COLUMN POSITIONS

```
' Old column layout: A=Section, B=Description, C=Action checkbox, D=Risk checkbox,  
' E=Decision, F=Owner, G=Due, H=Status, I=Priority, J=OD Days, K=Notes, L=Project hidden  
'  
' New column layout (Option 3): A=Type, B=Description, C=Owner, D=Due, E=Status,  
' F=Priority, G=OD Days, H=Notes, I=Project hidden  
'  
' Old format macros will write fills/CF to wrong columns. Do not run on current file.
```

BuildSummaryTab

Old Summary tab — read counts from AR. AR no longer exists as separate sheet, so this needs full rewrite for Option 3 if Summary tab is rebuilt later.

REFERENCES OLD AR SHEET

```
' Original code preserved if Summary tab is needed later.  
' Would need rewriting to: filter MM by Type=Action and Owner.  
' Replacement formulas (rough sketch):  
'   COUNTIFS('Meeting Minutes'!A16:A1000, "Action", 'Meeting Minutes'!C16:C1000, "TM1")  
'   COUNTIFS('Meeting Minutes'!A16:A1000, "Action", 'Meeting Minutes'!C16:C1000, "TM1", '  
' etc.
```

BuildHowToUseTab (v1)

Original How-To-Use builder. Refers to old structure (checkboxes, AR sheet, sync macros). Outdated — kept in Module 2 as v2 for current structure.

REFERENCES OLD STRUCTURE

```
' Use the v2 BuildHowToUseTab in Module 2 instead.  
' v1 references checkboxes, ACTION? column, sync workflow — none apply to Option 3.
```

Utilities — occasional one-off helpers

TestClearColCData

Sandbox test macro from Phase 4 — proved we could clear data in one column without breaking merged rows or header.

TEST/DIAGNOSTIC

```
Sub TestClearColCData()  
    Dim ws As Worksheet  
    Set ws = ThisWorkbook.Sheets("Meeting Minutes")  
    Dim lastRow As Long  
    lastRow = ws.Cells(ws.Rows.Count, 3).End(xlUp).Row  
    If lastRow < 16 Then  
        MsgBox "No data in col C below row 15."  
        Exit Sub  
    End If  
    Dim r As Long  
    Dim cleared As Long  
    For r = 16 To lastRow  
        If Not ws.Cells(r, 3).MergeCells Then  
            ws.Cells(r, 3).ClearContents  
            cleared = cleared + 1  
        End If  
    Next r  
    MsgBox "Cleared " & cleared & " cells in col C."  
End Sub
```

DiagnoseRows

Shows MergeCells state for the 10 rows below the active cell. Useful when banners look weird.

TEST/DIAGNOSTIC

```
Sub DiagnoseRows()  
    Dim ws As Worksheet  
    Dim r As Long  
    Dim msg As String  
    Set ws = Sheets("Meeting Minutes")  
    For r = ActiveCell.Row + 1 To ActiveCell.Row + 10  
        msg = msg & "Row " & r & ": MergeCells=" & ws.Cells(r, 1).MergeCells & Chr(10)  
    Next r  
    MsgBox msg  
End Sub
```

DiagnoseValidation

Prints existing data-validation state to Immediate window. Useful when dropdowns silently fail.

TEST/DIAGNOSTIC

```
Sub DiagnoseValidation()  
    Dim wsAR As Worksheet  
    Set wsAR = ThisWorkbook.Sheets("Action Register")  
    Debug.Print "Sheet name: [" & wsAR.Name & "]"  
    Debug.Print "Col Z row 1: [" & wsAR.Cells(1, 26).Value & "]"  
    Debug.Print "Col Z row 2: [" & wsAR.Cells(2, 26).Value & "]"  
    On Error Resume Next  
    Dim v As String  
    v = wsAR.Range("A4").Validation.Formula1  
    Debug.Print "Current A4 validation: [" & v & "]"  
    On Error GoTo 0  
    Debug.Print "Sheet protected: " & wsAR.ProtectContents  
End Sub
```